

图像分类流程

图像分类: 计算机视觉的一个核心任务



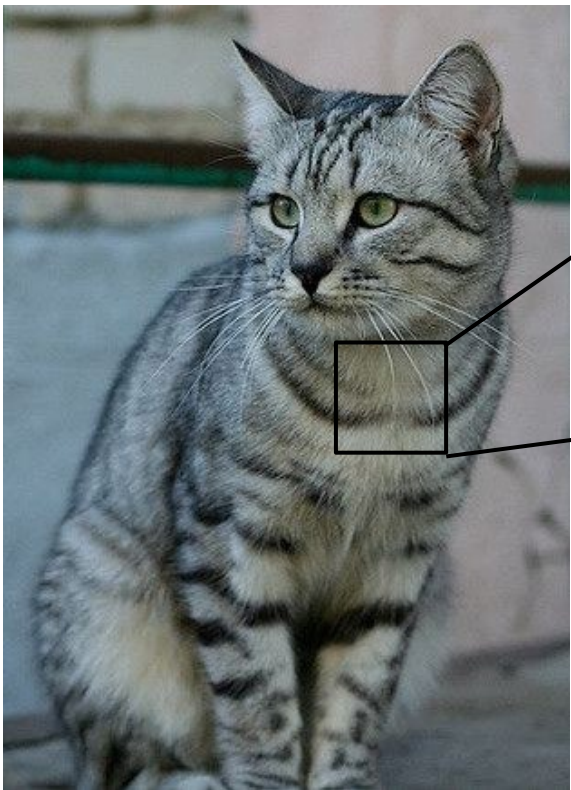
[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

(假设给定一组离散标签)
{狗、猫、卡车、飞机.....}



猫

Challenges: Semantic Gap (语义鸿沟)



[This image](#) by [Nikita](#) is
licensed under [CC-BY 2.0](#)

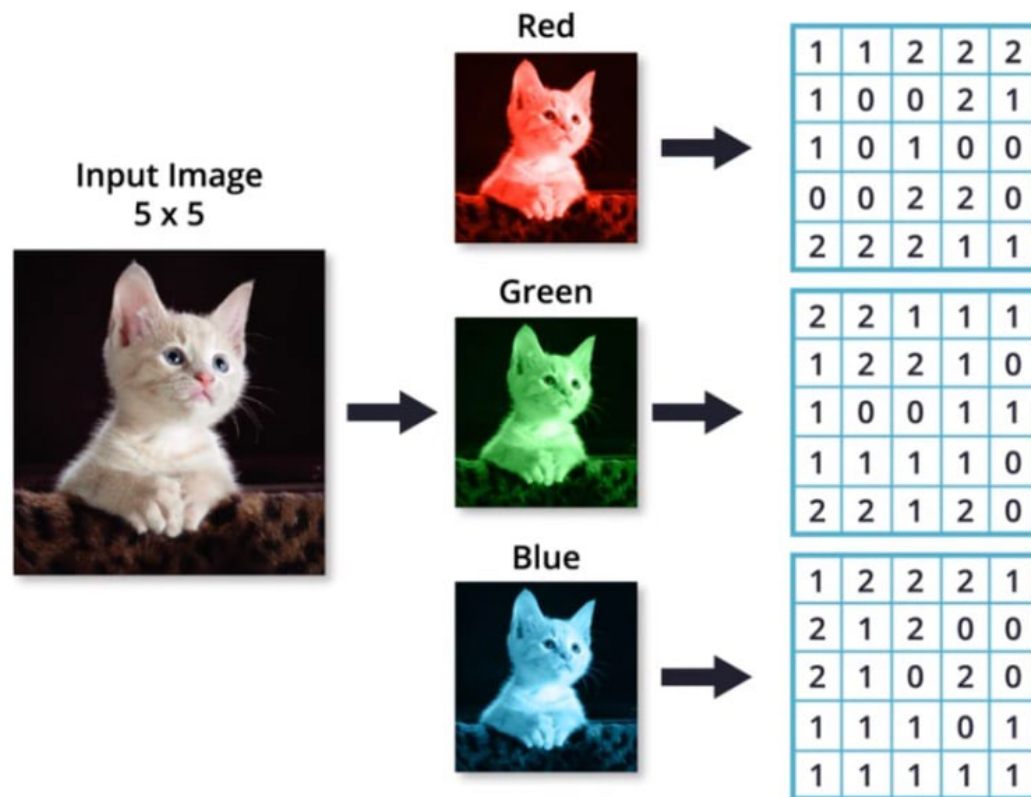
[	105	112	108	111	104	99	106	99	96	103	112	119	104	97	93	87]
[	91	98	102	106	104	79	98	103	99	105	123	136	110	105	94	85]
[	76	85	90	105	128	105	87	96	95	99	115	112	106	103	99	85]
[	99	81	81	93	120	131	127	100	95	98	102	99	96	93	101	94]
[	106	91	61	64	69	91	88	85	101	107	109	98	75	84	96	95]
[	114	108	85	55	55	69	64	54	64	87	112	129	98	74	84	91]
[	133	137	147	103	65	81	80	65	52	54	74	84	102	93	85	82]
[	128	137	144	140	109	95	86	70	62	65	63	63	60	73	86	101]
[	125	133	148	137	119	121	117	94	65	79	80	65	54	64	72	98]
[	127	125	131	147	133	127	126	131	111	96	89	75	61	64	72	84]
[	115	114	109	123	150	148	131	118	113	109	100	92	74	65	72	78]
[	89	93	90	97	108	147	131	118	113	114	113	109	106	95	77	80]
[	63	77	86	81	77	79	102	123	117	115	117	125	125	130	115	87]
[	62	65	82	89	78	71	80	101	124	126	119	101	107	114	131	119]
[	63	65	75	88	89	71	62	81	120	138	135	105	81	98	110	118]
[	87	65	71	87	106	95	69	45	76	130	126	107	92	94	105	112]
[	118	97	82	86	117	123	116	66	41	51	95	93	89	95	102	107]
[	164	146	112	80	82	120	124	104	76	48	45	66	88	101	102	109]
[	157	170	157	120	93	86	114	132	112	97	69	55	70	82	99	94]
[	130	128	134	161	139	100	109	118	121	134	114	87	65	53	69	86]
[	128	112	96	117	150	144	120	115	104	107	102	93	87	81	72	79]
[	123	107	96	86	83	112	153	149	122	109	104	75	80	107	112	99]
[	122	121	102	80	82	86	94	117	145	148	153	102	58	78	92	107]
[	122	164	148	103	71	56	78	83	93	103	119	139	102	61	69	84]

计算机看到的

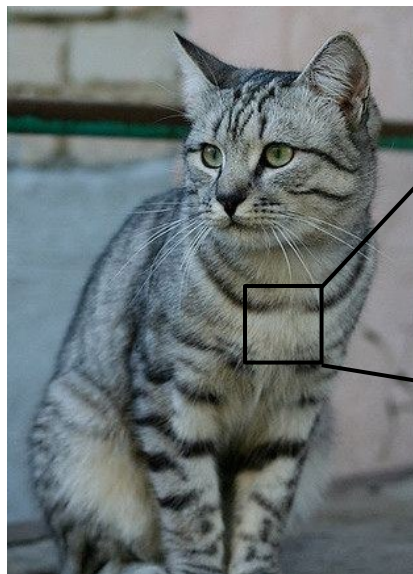
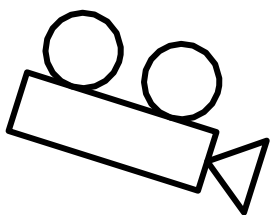
一张图像就是一个由[0,255]
之间的数字组成的大网格

e.g. 800 x 600 x 3 (3
个通道的 RGB)

RGB 彩色图片



Challenges: viewpoint variation (视角变化)



```
[105 112 108 111 104 99 106 99 96 103 112 119 104 97 93 87]  
[ 91 98 102 106 104 79 98 103 99 105 123 136 110 105 94 85]  
[ 76 85 90 105 128 105 87 96 95 99 115 112 106 103 99 85]  
[ 99 81 81 93 120 131 127 100 95 98 102 99 96 93 101 94]  
[106 91 61 64 69 91 88 85 101 107 109 98 75 84 96 95]  
[114 108 85 55 55 69 64 54 64 87 112 129 98 74 84 91]  
[133 137 147 103 65 81 80 65 52 54 74 84 102 93 85 82]  
[128 137 144 140 109 95 86 70 62 65 63 63 60 73 86 101]  
[125 133 148 137 119 121 117 94 65 79 80 65 54 64 72 98]  
[127 125 131 147 133 127 126 131 111 96 89 75 61 64 72 84]  
[115 114 109 123 150 148 131 118 113 109 100 92 74 65 72 78]  
[ 89 93 90 97 108 147 131 118 113 114 113 109 106 95 77 80]  
[ 63 77 86 81 77 79 102 123 117 115 117 125 125 130 115 87]  
[ 62 65 82 89 78 71 80 101 124 126 119 101 107 114 131 119]  
[ 63 65 75 88 89 71 62 81 120 138 135 105 81 98 110 118]  
[ 87 65 71 87 106 95 69 45 76 130 126 107 92 94 105 112]  
[118 97 82 86 117 123 116 66 41 51 95 93 89 95 102 107]  
[164 146 112 80 82 120 124 104 76 48 45 66 88 101 102 109]  
[157 170 157 120 93 86 114 132 112 97 69 55 70 82 99 94]  
[130 128 134 161 139 100 109 118 121 134 114 87 65 53 69 86]  
[128 112 96 117 150 144 120 115 104 107 102 93 87 81 72 79]  
[123 107 96 86 83 112 153 149 122 109 104 75 80 107 112 99]  
[122 121 102 80 82 86 94 117 145 148 153 102 58 78 92 107]  
[122 164 148 103 71 56 78 83 93 103 119 139 102 61 69 84]
```

当相机移动时，
所有像素都会改变！

Challenges: Background Clutter (背景雑波)



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

Challenges: Illumination (照明)



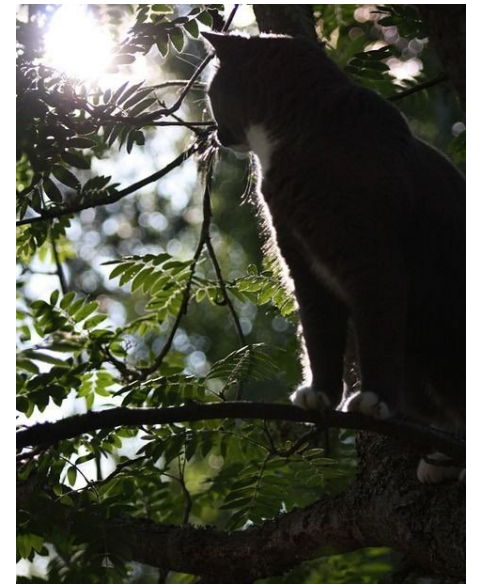
[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain

Challenges: Deformation (变形)



[This image](#) by [Umberto Salvagnin](#)
is licensed under [CC-BY 2.0](#)



[This image](#) by [Umberto Salvagnin](#)
is licensed under [CC-BY 2.0](#)



[This image](#) by [sare bear](#) is
licensed under [CC-BY 2.0](#)



[This image](#) by [Tom Thai](#) is
licensed under [CC-BY 2.0](#)

Challenges: Occlusion (遮挡)



[This image](#) is [CC0 1.0](#) public domain



[This image](#) is [CC0 1.0](#) public domain



[This image](#) by [jonsson](#) is licensed under [CC-BY 2.0](#)

Challenges: Intraclass variation (类内变化)



[This image](#) is [CC0 1.0](#) public domain

图像分类器

```
def classify_image(image):  
    # Some magic here?  
    return class_label
```

不同于，例如，对一串数字排序

没有明显的方法来硬编码识别猫或其他类的算法。

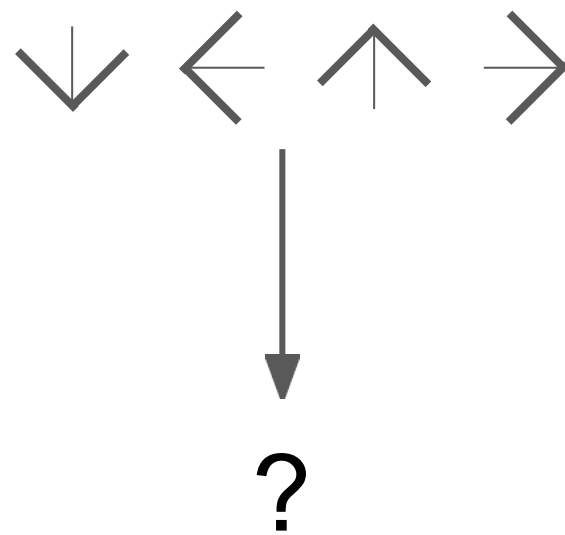
此前有过尝试



圈定边缘



寻找角



机器学习:数据驱动的方法

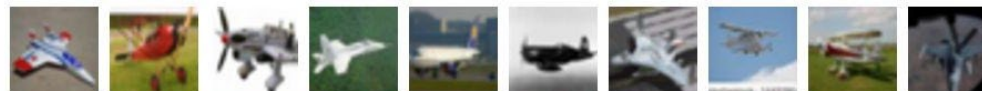
1. 收集数据集, 包含图像和标签
2. 使用机器学习训练分类器
3. 对新图像评估分类器

```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

训练集样本

airplane



automobile



bird



cat



deer



第一个分类器: **Nearest Neighbor** (最近邻)

```
def train(images, labels):  
    # Machine learning!  
    return model
```



记住所有的数据
和标签

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```



用最相似的训练图
像的标签预测

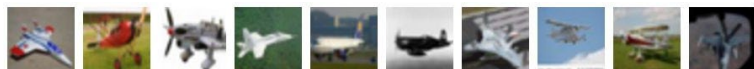
Example Dataset: **CIFAR10**

10 个类别

50,000 张训练图片

10,000 张测试图片

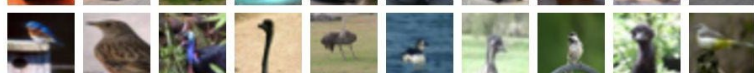
airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



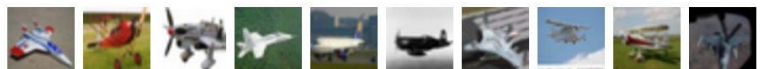
Example Dataset: **CIFAR10**

10 个类别

50,000 张训练图片

10,000 张测试图片

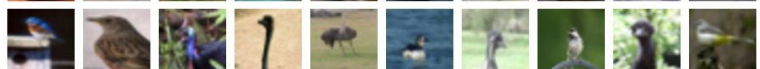
airplane



automobile



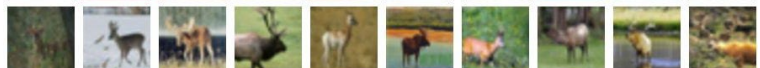
bird



cat



deer



dog



frog



horse



ship



truck



测试图像和最近邻



距离度量来比较图像

L1 距离:

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

测试图片
test image

56	32	10	18
90	23	128	133
24	26	178	200
2	0	255	220

训练图片
training image

10	20	24	17
8	10	89	100
12	16	178	170
4	32	233	112

-

=

像素级的绝对值差

pixel-wise absolute value differences

46	12	14	1
82	13	39	33
12	10	0	30
2	32	22	108

add
→ 456

最近邻分类器

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):
```

```
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
```

```
        # the nearest neighbor classifier simply remembers all the training data
```

```
        self.Xtr = X
```

```
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """
```

```
        num_test = X.shape[0]
```

```
        # lets make sure that the output type matches the input type
```

```
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)
```

```
        # loop over all test rows
```

```
        for i in xrange(num_test):
```

```
            # find the nearest training image to the i'th test image
```

```
            # using the L1 distance (sum of absolute value differences)
```

```
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
```

```
            min_index = np.argmin(distances) # get the index with smallest distance
```

```
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
        return Ypred
```

最近邻分类器

记住训练数据

最近邻分类器

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

        # loop over all test rows
        for i in xrange(num_test):
            # find the nearest training image to the i'th test image
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

对于每个测试图像:
找到距离最近的训练图像
用最近图像的标签来预测

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):
```

```
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
```

```
        # the nearest neighbor classifier simply remembers all the training data
```

```
        self.Xtr = X
```

```
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """
```

```
        num_test = X.shape[0]
```

```
        # lets make sure that the output type matches the input type
```

```
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)
```

```
        # loop over all test rows
```

```
        for i in xrange(num_test):
```

```
            # find the nearest training image to the i'th test image
```

```
            # using the L1 distance (sum of absolute value differences)
```

```
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
```

```
            min_index = np.argmin(distances) # get the index with smallest distance
```

```
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
        return Ypred
```

最近邻分类器

Q:对于N个样本，训练和预测的速度有多快

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):
```

```
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
```

```
        # the nearest neighbor classifier simply remembers all the training data
```

```
        self.Xtr = X
```

```
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """
```

```
        num_test = X.shape[0]
```

```
        # lets make sure that the output type matches the input type
```

```
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)
```

```
        # loop over all test rows
```

```
        for i in xrange(num_test):
```

```
            # find the nearest training image to the i'th test image
```

```
            # using the L1 distance (sum of absolute value differences)
```

```
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
```

```
            min_index = np.argmin(distances) # get the index with smallest distance
```

```
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
        return Ypred
```

最近邻分类器

Q:对于N个样本，训练和预测的速度有多快

A: 训练 $O(1)$,
预测 $O(N)$

```
import numpy as np
```

```
class NearestNeighbor:
```

```
    def __init__(self):  
        pass
```

```
    def train(self, X, y):
```

```
        """ X is N x D where each row is an example. Y is 1-dimension of size N """  
        # the nearest neighbor classifier simply remembers all the training data  
        self.Xtr = X  
        self.ytr = y
```

```
    def predict(self, X):
```

```
        """ X is N x D where each row is an example we wish to predict label for """  
        num_test = X.shape[0]  
        # lets make sure that the output type matches the input type  
        Ypred = np.zeros(num_test, dtype = self.ytr.dtype)
```

```
        # loop over all test rows
```

```
        for i in xrange(num_test):
```

```
            # find the nearest training image to the i'th test image
```

```
            # using the L1 distance (sum of absolute value differences)
```

```
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
```

```
            min_index = np.argmin(distances) # get the index with smallest distance
```

```
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
```

```
    return Ypred
```

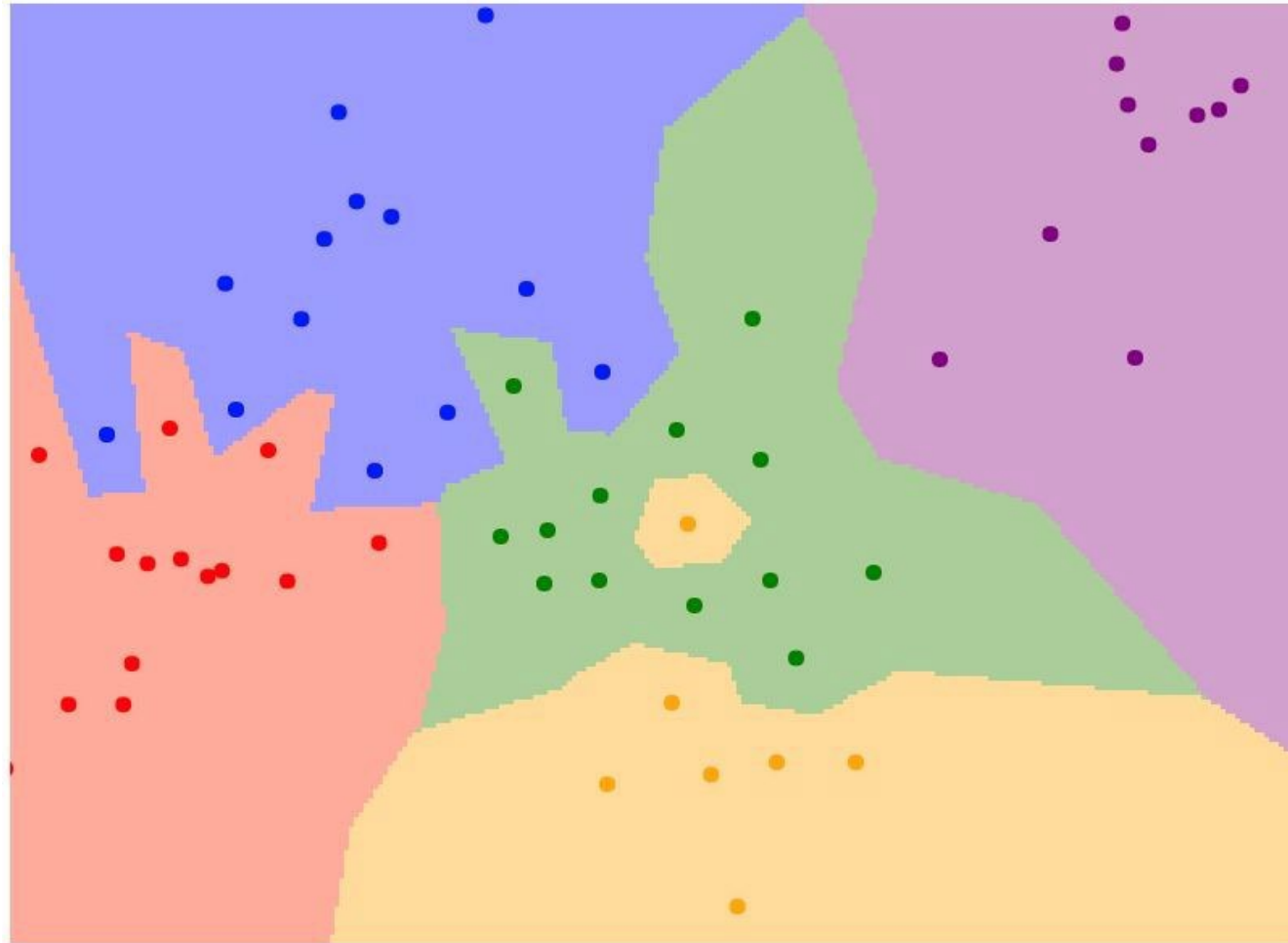
最近邻分类器

Q:对于N个样本，训练和预测的速度有多快

A: 训练 $O(1)$,
预测 $O(N)$

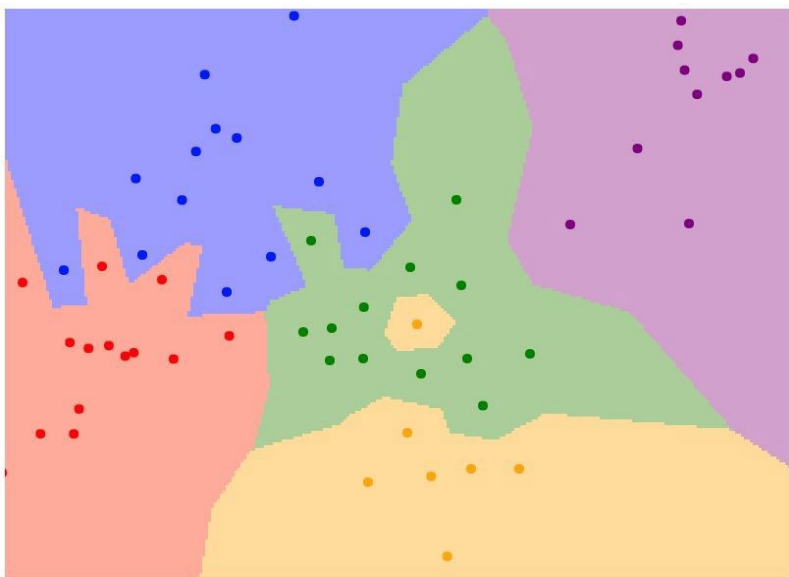
这很糟糕：我们想要在预测中**快速**的分类器；
训练**慢**是可以的

What does this look like?

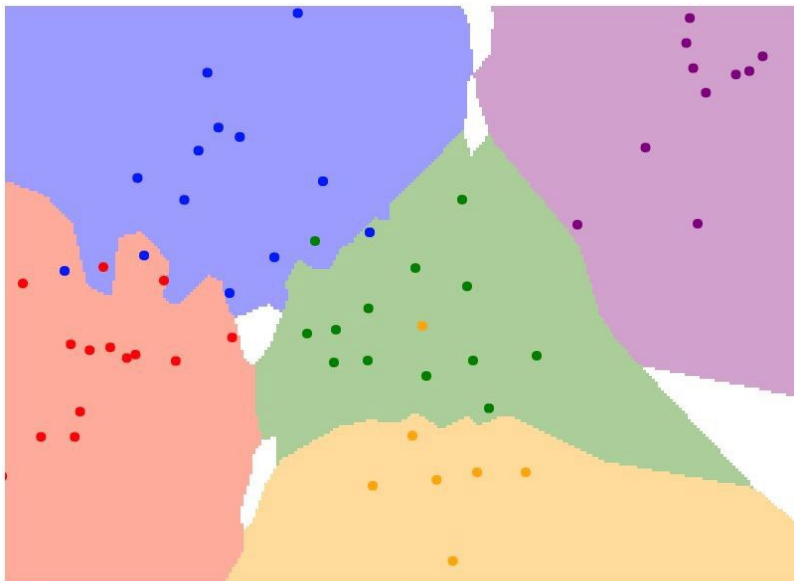


K-最近邻

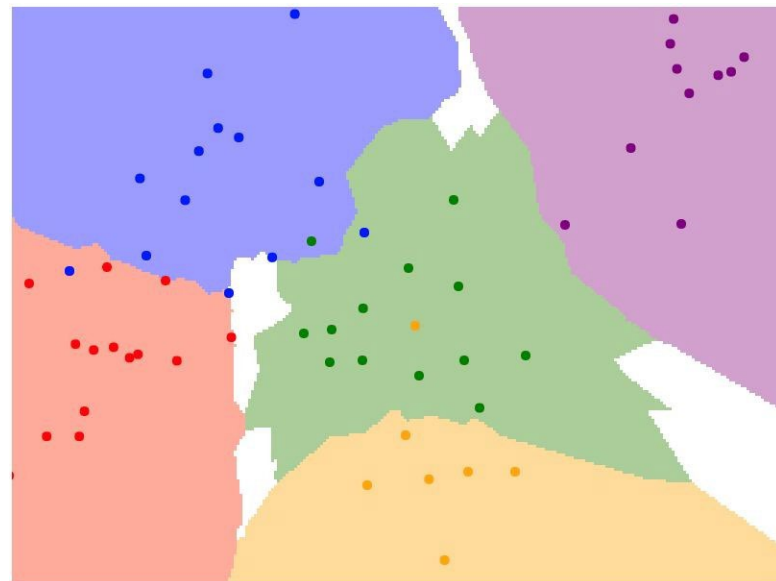
不是从最近的邻居直接复制标签，
而是从K个最近点中进行**多数投票**来决定标签



$K = 1$



$K = 3$



$K = 5$

What does this look like?



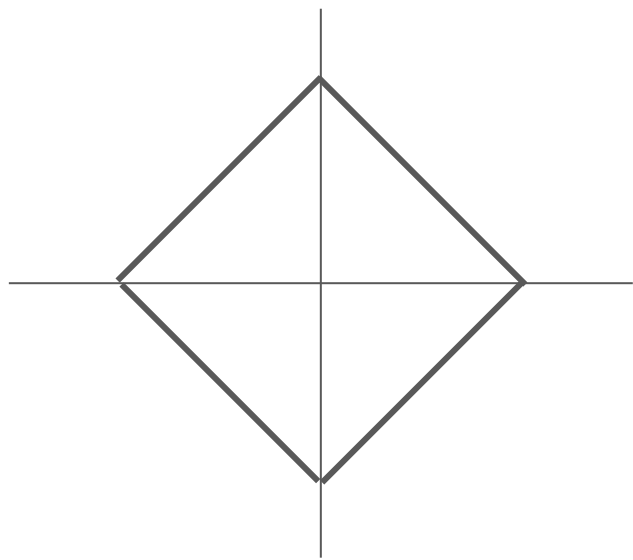
What does this look like?



K-最近邻: 距离度量

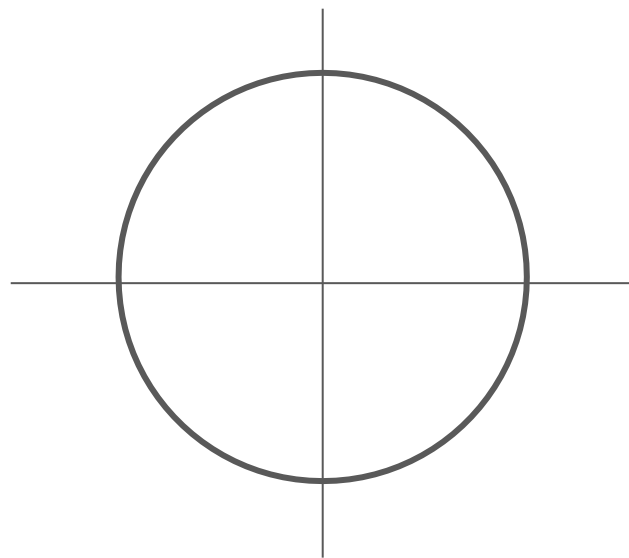
L1 (曼哈顿) 距离

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



L2 (欧几里得) 距离

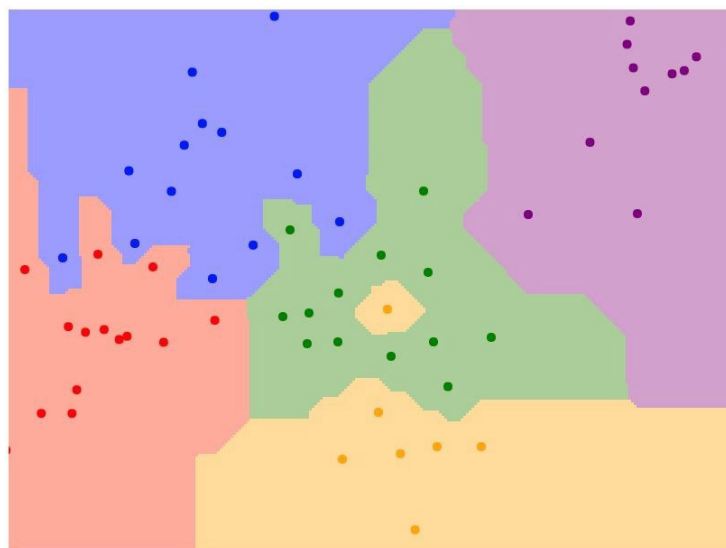
$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K-最近邻: 距离度量

L1 (曼哈顿) 距离

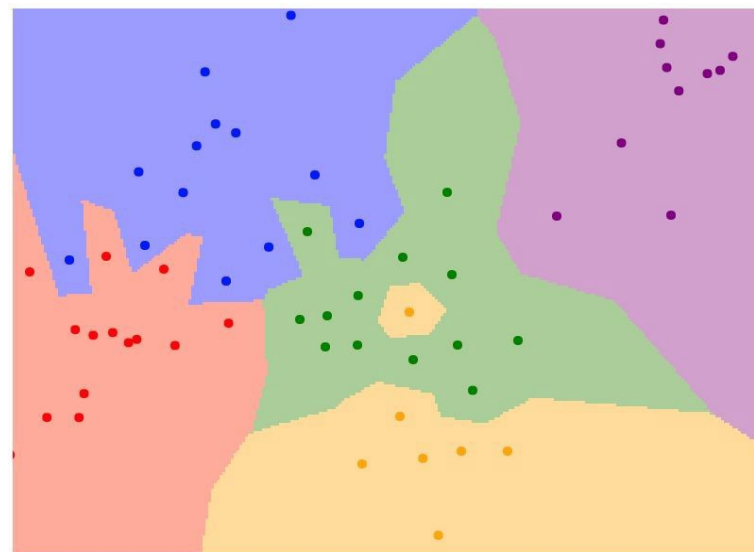
$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

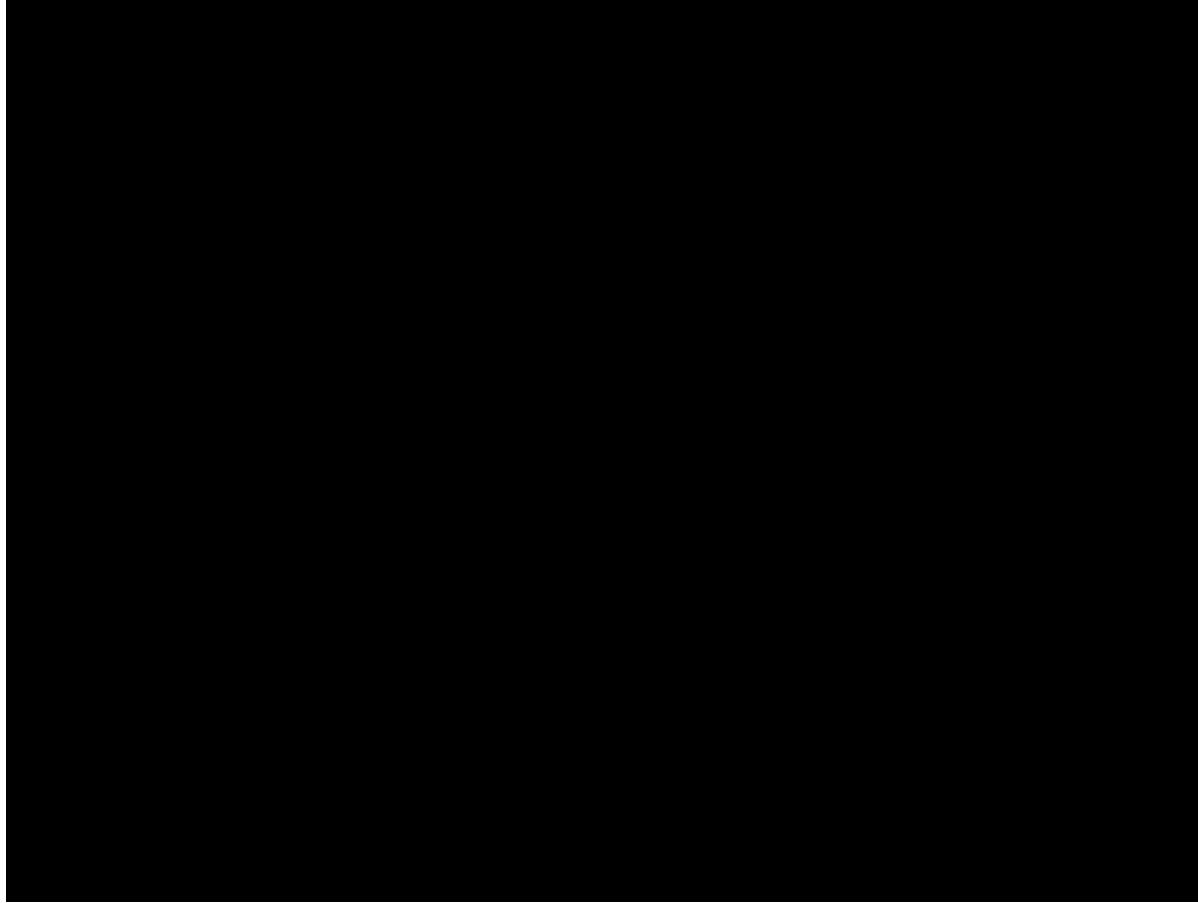
L2 (欧几里得) 距离

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K = 1

K-Nearest Neighbors: Demo Time



<http://vision.stanford.edu/teaching/cs231n-demos/knn/>

超参数

最佳的**k**值是多少？

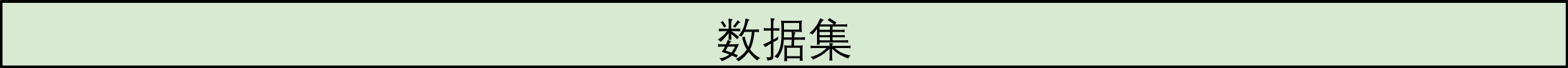
最好的**距离度量**是多少？

这些是**超参数**:关于算法的选择是我们设定的，而不是学习的

这非常依赖于具体问题。必须尝试所有方法，看看哪一种最有效。

设定超参数

Idea #1:选择对数据最有效的超参数



数据集

设定超参数

当我们在最近邻算法 (k-NN) 中使用 $K=1$ 时, 对于每个训练数据点, 其最近的邻居总是它自己。

Idea #1:选择对数据最有效的超参数

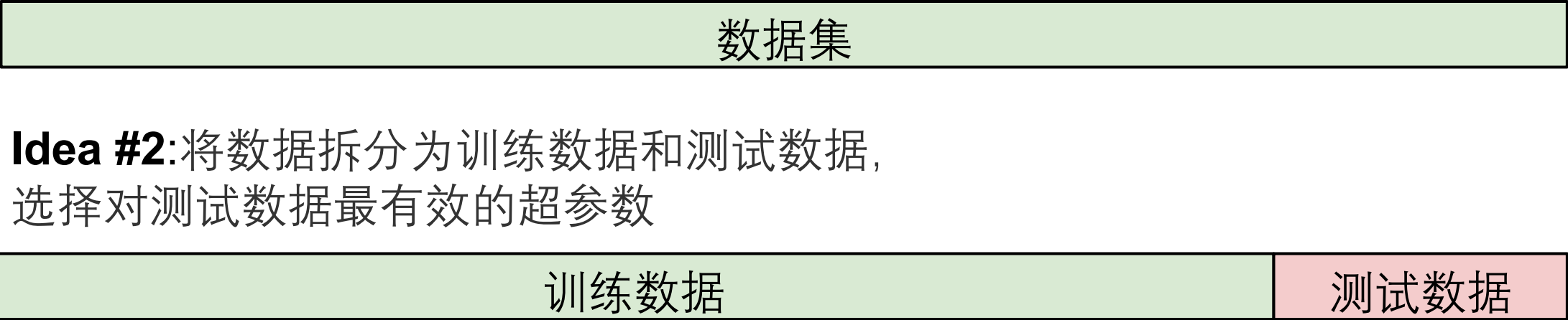
**BAD: $K = 1$ 对于训练数据
总是非常有效**

数据集

设定超参数

Idea #1:选择对数据最有效的超参数

BAD: $K = 1$ 对于训练数据总是非常有效



数据集

Idea #2:将数据拆分为训练数据和测试数据,
选择对测试数据最有效的超参数

训练数据

测试数据

设定超参数

Idea #1:选择对数据最有效的超参数

BAD: $K = 1$ 对于训练数据总是非常有效

数据集

Idea #2:将数据拆分为训练数据和测试数据,选择对测试数据最有效的超参数

BAD:不知道算法在新数据上的表现如何

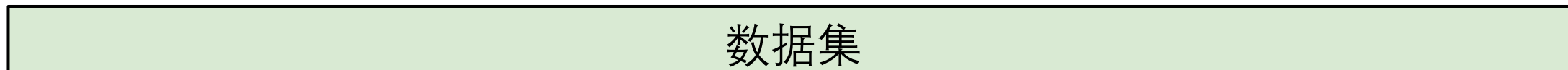
训练数据

测试数据

设定超参数

Idea #1:选择对数据最有效的超参数

BAD: $K = 1$ 对于训练数据总是非常有效



Idea #2:将数据拆分为训练数据和测试数据,选择对测试数据最有效的超参数

BAD:不知道算法在新数据上的表现如何



Idea #3:将数据拆分为训练集、验证集和测试集;在验证集上选择超参数,在测试集上评估

Better!



设定超参数

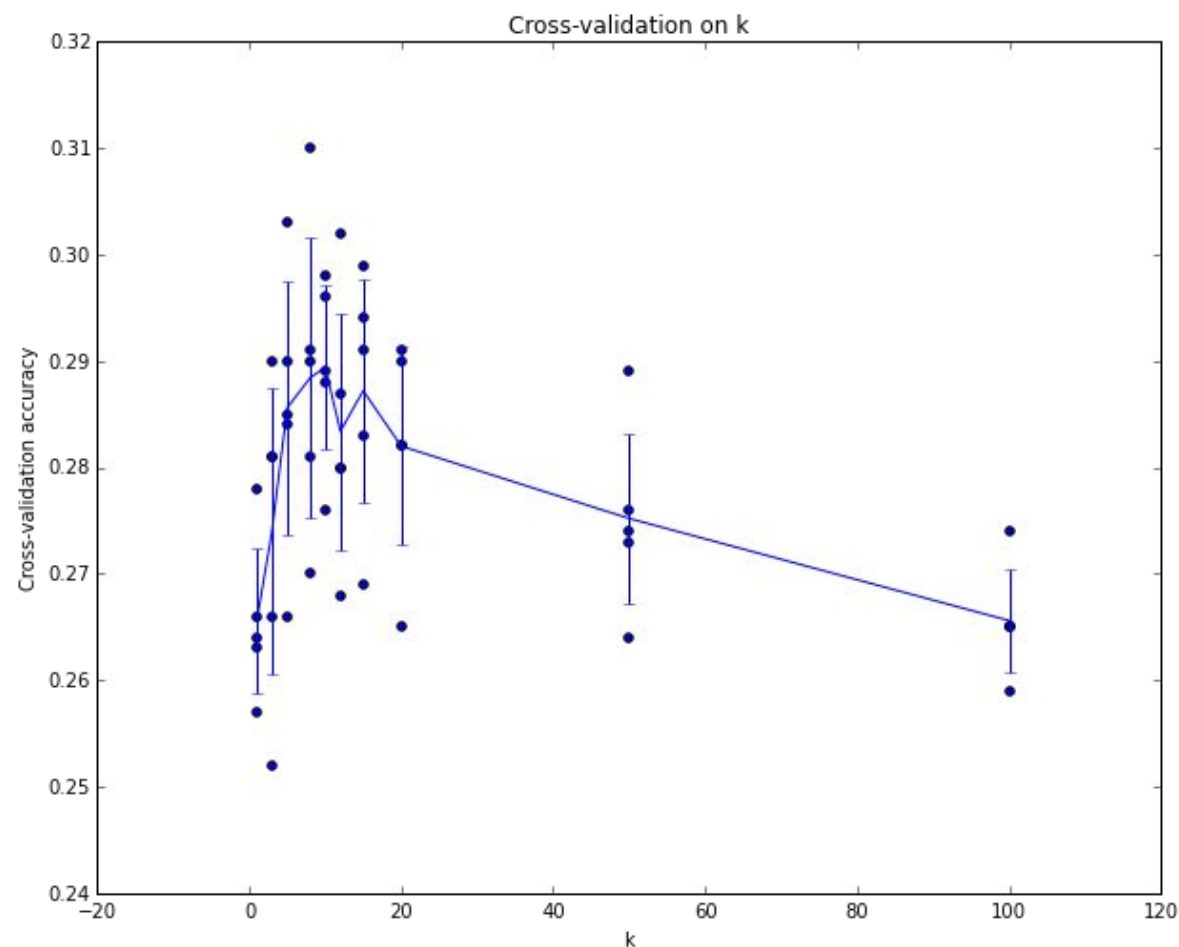
数据集

Idea #4:交叉验证:将数据分成几份，尝试每一份依次作为验证，并计算结果的平均值

fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test
fold 1	fold 2	fold 3	fold 4	fold 5	test

对小数据集有用，但在深度学习中不常用

设定超参数



5折交叉验证用于k的值。

每个点：单一的结果。

线条穿越均值，条形表示标准偏差。

(对于这些数据，k大约等于7似乎是最好的)

在图像上从未采用过k-最近邻算法。

- 测试时非常慢
- 基于像素的距离度量不具有信息性。

原图



黑框遮挡



平移



着色



(所有3张图像与左侧图像的L2距离相同)

K-最近邻: 总结

在**图像分类**中，我们从一组**训练图像和标签**开始，必须在**测试集**上预测标签。

K-最近邻分类器基于距离最近的训练示例预测标签。

距离度量和K都是**超参数**。

使用**验证集**选择超参数；只在最后一步在测试集上运行一次！

案例：多特征图像分类的K-NN实践

- 使用数据集和多个特征，通过手动计算，应用K-NN算法进行图像分类。
- 尝试不同的K值和距离度量方法，预测类别

案例：多特征图像分类的K-NN实践

- 训练集
 - 假设数据集包含图像，每个图像代表3种类别中的一种（例如，类别A、类别B、类别C），且每个图像由4个特征描述：特征1 (X1)，特征2 (X2)，特征3 (X3)，特征4 (X4)。

图像编号	X1	X2	X3	X4	类别
1	1	1	2	2	A
2	2	1	2	3	A
3	4	3.5	4	4.5	B
4	5	4	5	5	B
5	1	2	1	2	A
6	2	2.5	2	2	C
7	4.5	5	5	5.5	B
8	3	2.5	3	3.5	C
9	3	3	3	3	C
10	5	5.5	6	6	B

案例：多特征图像分类的K-NN实践

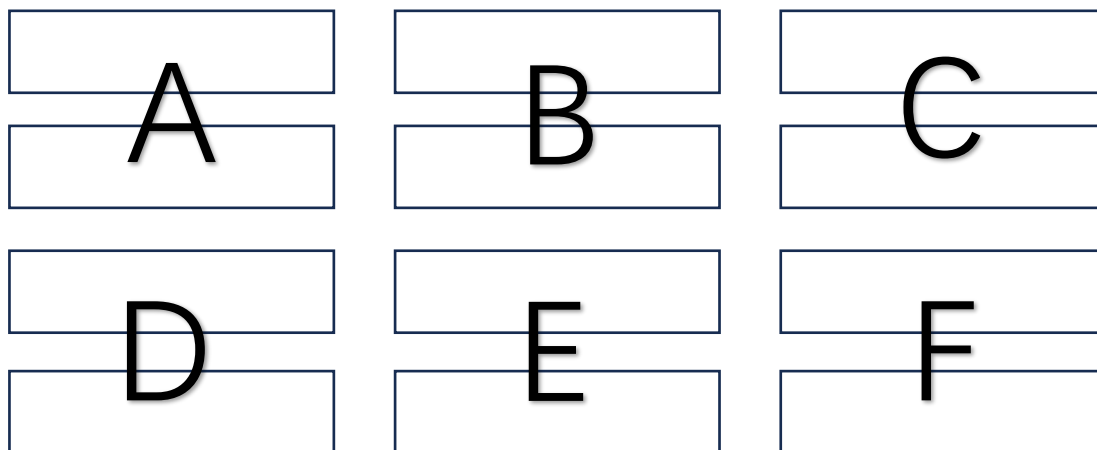
- 测试集

测试图像编号	X1	X2	X3	X4
T1	3	3	2.5	3
T2	2	1.5	2	2
T3	4.5	4	4.5	5

案例：多特征图像分类的K-NN实践

- 任务说明：

- 每组选择不同的K值（例如， $K=1$, $K=3$, $K=5$ ）和不同的距离度量方法（例如，欧氏距离、曼哈顿距离）。
- 计算完成后展示分类结果和使用的方法



组A :K值: 1,距离度量: 欧氏距离
组B: K值: 3, 距离度量: 欧氏距离
组C :K值: 5, 距离度量: 欧氏距离
组D :K值: 1, 距离度量: 曼哈顿距离
组E : K值: 3, 距离度量: 曼哈顿距离
组F : K值: 5, 距离度量: 曼哈顿距离

案例：多特征图像分类的K-NN实践

欧氏距离

[illegible]

曼哈顿距离

测试 图像	训练 图像1	训练 图像2	训练 图像3	训练 图像4	训练 图像5	训练 图像 6	训练 图像7	训练 图像 8	训练 图像9	训练 图像 10
T1										
T2										
T3										

案例：多特征图像分类的K-NN实践

- 测试集实际类别

测试图像编号	预测类别
T1	C
T2	A
T3	B

- 你们组的K-NN模型预测对了吗？

线性分类

线性代数基本概念

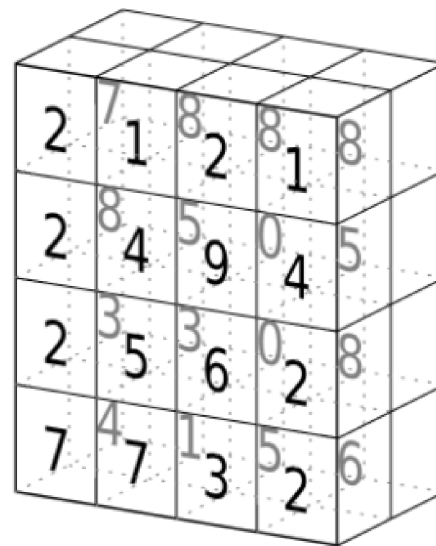
1

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$$

- Scalar (标量)
 - 单一的数或只有一个元素的矩阵
- Vector (向量)
 - 向量是一维数字数组
- Matrix (矩阵)
 - 2维数组，其中每个元素由两个索引(行和列)标识。
- Tensor (张量)
 - 一个n维数组



矩阵转置

- 向量转置

- 行向量 → 列向量

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 \end{bmatrix}$$

- 矩阵转置

- 形状为 $(N \times M)$ 的矩阵转置后变为 $(M \times N)$

$$\mathbf{A} = \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix}$$

$$\mathbf{A}^T = \begin{bmatrix} A_{1,1} & A_{2,1} & A_{3,1} \\ A_{1,2} & A_{2,2} & A_{3,2} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \quad \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix}^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$$

矩阵相加

- 同样形状的矩阵可以进行相加，相加后的矩阵形状也相同
- 每个对应元素进行相加

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} + \begin{bmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \\ B_{3,1} & B_{3,2} \end{bmatrix} = \begin{bmatrix} A_{1,1} + B_{1,1} & A_{1,2} + B_{1,2} \\ A_{2,1} + B_{2,1} & A_{2,2} + B_{2,2} \\ A_{3,1} + B_{3,1} & A_{3,2} + B_{3,2} \end{bmatrix}$$

矩阵和标量相加

$$\alpha + \begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} = \begin{bmatrix} \alpha + A_{1,1} & \alpha + A_{1,2} \\ \alpha + A_{2,1} & \alpha + A_{2,2} \\ \alpha + A_{3,1} & \alpha + A_{3,2} \end{bmatrix}$$

Broadcasting 广播

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} + \begin{bmatrix} B_{1,1} \\ B_{2,1} \\ B_{3,1} \end{bmatrix}$$

相当于

$$\begin{bmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \\ A_{3,1} & A_{3,2} \end{bmatrix} + \begin{bmatrix} B_{1,1} & B_{1,1} \\ B_{2,1} & B_{2,1} \\ B_{3,1} & B_{3,1} \end{bmatrix} = \begin{bmatrix} A_{1,1} + B_{1,1} & A_{1,2} + B_{1,1} \\ A_{2,1} + B_{2,1} & A_{2,2} + B_{2,1} \\ A_{3,1} + B_{3,1} & A_{3,2} + B_{3,1} \end{bmatrix}$$

矩阵乘法

- 计算行和列的乘积之和, 而**不是**每个元素相乘
- 第一个矩阵的形状是($m \times n$), 第二个矩阵的形状必须为($n \times k$), 乘积矩阵的形状为($m \times k$)
- 第一个矩阵的列数是第二个矩阵的行数

The diagram illustrates the process of matrix multiplication. On the left, a 3×2 matrix is shown with elements A, B in the first row, C, D in the second row, and E, F in the third row. Below it, the dimensions 3×2 are written. To its right is a 2×1 matrix with elements G in the first row and H in the second row. Below it, the dimensions 2×1 are written. A blue arrow points from the first row of the first matrix to the first row of the result matrix, and an orange arrow points from the second row of the first matrix to the second row of the result matrix. An equals sign follows the second matrix. To the right of the equals sign is the resulting 3×1 matrix, which contains the sums of products: $A \times G + B \times H$ in the first row, $C \times G + D \times H$ in the second row, and $E \times G + F \times H$ in the third row. Below the result matrix, the dimensions 3×1 are written. A blue arrow also points from the first row of the first matrix to the first row of the result matrix, and an orange arrow points from the second row of the first matrix to the second row of the result matrix.

$$\begin{bmatrix} A & B \\ C & D \\ E & F \end{bmatrix} \times \begin{bmatrix} G \\ H \end{bmatrix} = \begin{bmatrix} A \times G + B \times H \\ C \times G + D \times H \\ E \times G + F \times H \end{bmatrix}$$

3×2 2×1 3×1

矩阵乘法

$$\text{矩阵 } \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \text{ 和 向量 } \mathbf{b} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

矩阵乘法

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \times \begin{bmatrix} 2 \\ 4 \end{bmatrix} =$$

$$\begin{bmatrix} 1 \times 2 + 2 \times 4 \\ 3 \times 2 + 4 \times 4 \\ 5 \times 2 + 6 \times 4 \end{bmatrix} = \begin{bmatrix} 10 \\ 22 \\ 34 \end{bmatrix}$$

<http://matrixmultiplication.xyz/>

矩阵乘法

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} 2 & 7 \\ 1 & 2 \\ 3 & 6 \end{bmatrix}$$

<http://matrixmultiplication.xyz/>

矩阵乘法

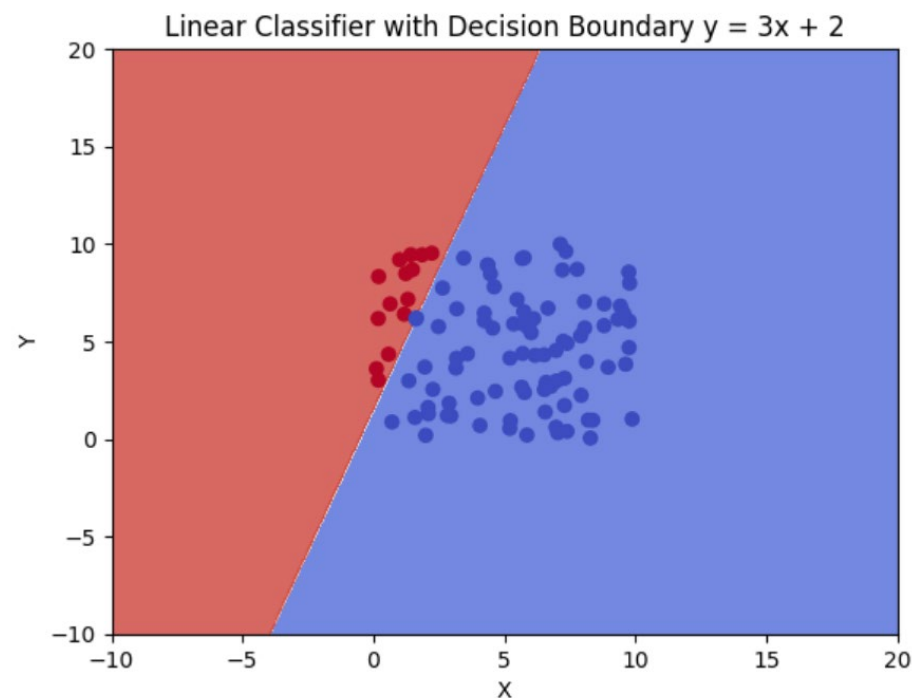
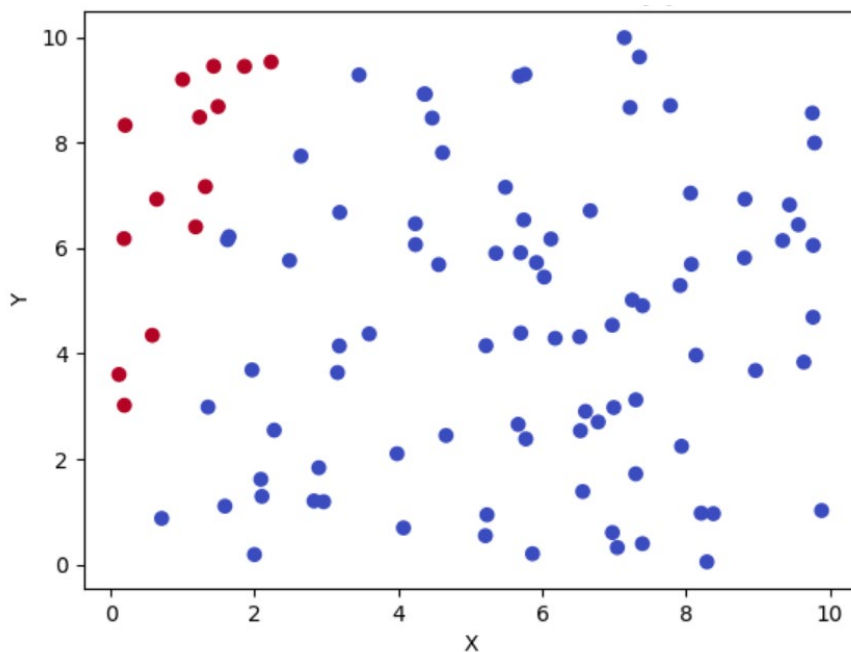
$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \\ 10 & 11 & 12 \end{bmatrix} \times \begin{bmatrix} 2 & 7 \\ 1 & 2 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} 2 \times 1 + 1 \times 2 + 3 \times 3 & 7 \times 1 + 2 \times 2 + 6 \times 3 \\ 2 \times 4 + 1 \times 5 + 3 \times 6 & 7 \times 4 + 2 \times 5 + 6 \times 6 \\ 2 \times 7 + 1 \times 8 + 3 \times 9 & 7 \times 7 + 2 \times 8 + 6 \times 9 \\ 2 \times 10 + 1 \times 11 + 3 \times 12 & 7 \times 10 + 2 \times 11 + 6 \times 12 \end{bmatrix} = \begin{bmatrix} 13 & 29 \\ 31 & 74 \\ 49 & 119 \\ 67 & 164 \end{bmatrix}$$

线性分类

- $y = 3x + 2$

$x_2 = 3x_1 + 2$

$3x_1 - x_2 + 2 = 0$ $\quad \quad \quad = \quad \quad \quad [3, -1] \times \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [2] = 0$



线性分类

- $y = 1x_1 + 2x_2 + 2$

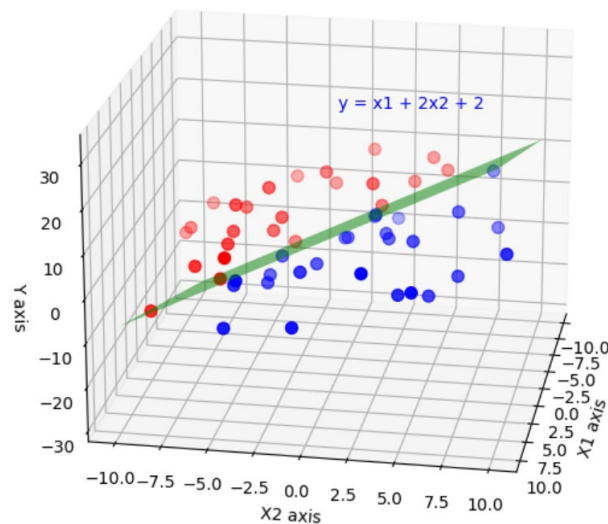


$$x_3 = 1x_1 + 2x_2 + 2$$



$$1x_1 + 2x_2 - x_3 + 2 = 0 \quad = \quad [1, 2, -1] \times \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + [2] = 0$$

- 图像:



<https://colab.research.google.com/drive/10fzCgKQbNDwuyapzygO8mFZIPDDt5wBX?usp=sharing>

使用矩阵来描述线性分类

矩阵可用于描述形式 $\mathbf{Ax} = \mathbf{b}$ 的线性方程系统。有这样一个系统：

$$A_{1,1}x_1 + A_{1,2}x_2 + A_{1,n}x_n = b_1$$

$$A_{2,1}x_1 + A_{2,2}x_2 + A_{2,n}x_n = b_2$$

...

$$A_{m,1}x_1 + A_{m,2}x_2 + A_{m,n}x_n = b_n$$

$$\begin{bmatrix} A_{1,1} & A_{1,2} & \cdots & A_{1,n} \\ A_{2,1} & A_{2,2} & \cdots & A_{2,n} \\ \cdots & \cdots & \cdots & \cdots \\ A_{m,1} & A_{m,2} & \cdots & A_{m,n} \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ \cdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \cdots \\ b_m \end{bmatrix}$$

神经网络

线性
分类器



[This image](#) is [CC0 1.0](#) public domain

两个小女孩正在玩
乐高玩具。



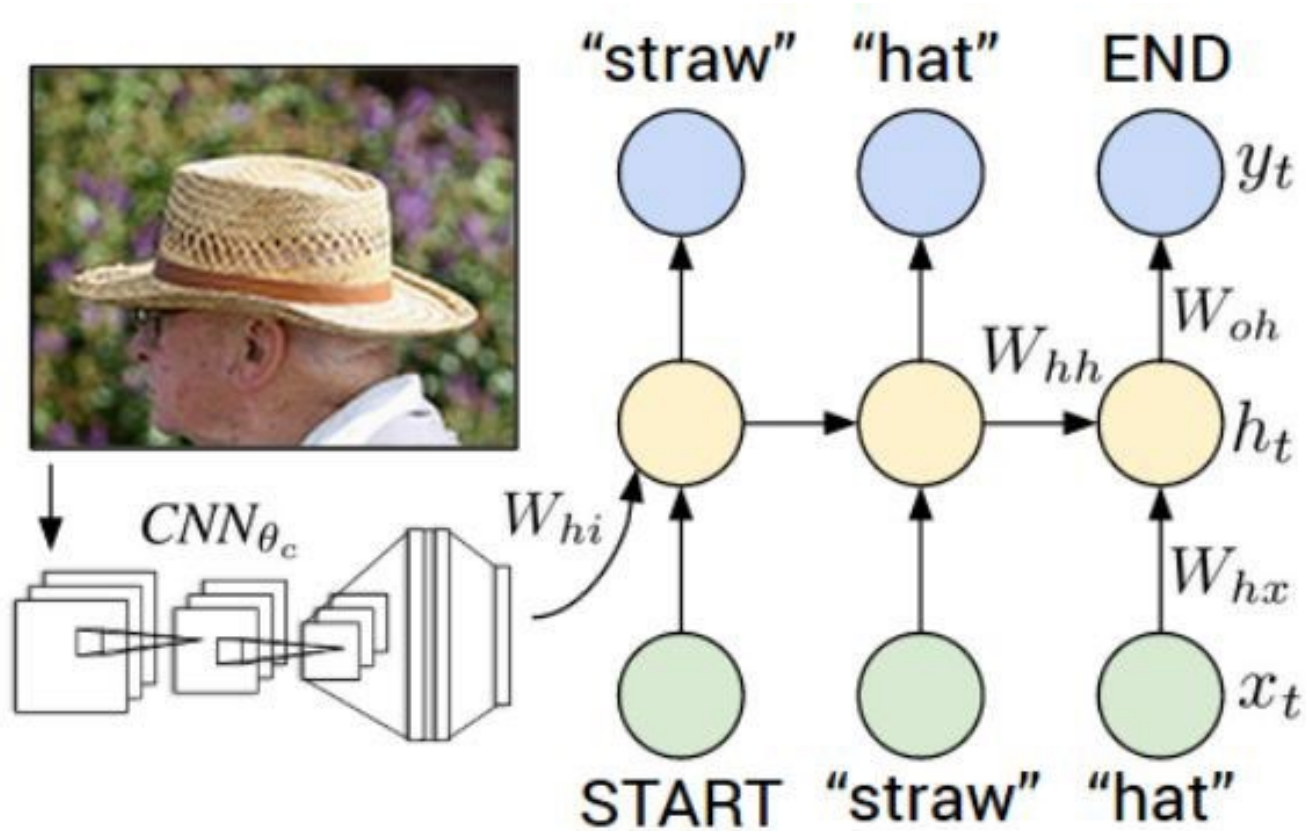
男孩正在用冲浪板做
后空翻。



穿黑色衬衫的男
人正在弹吉他。



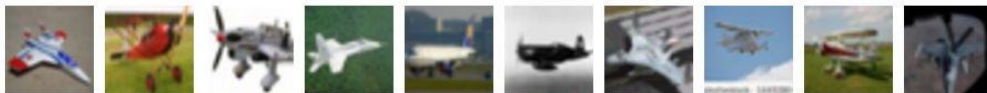
穿橙色安全背心的建筑
工人正在道路上工作。



Karpathy and Fei-Fei, "Deep Visual-Semantic Alignments for Generating Image Descriptions", CVPR 2015
Figures copyright IEEE, 2015. Reproduced for educational purposes.

回想CIFAR10

airplane



automobile



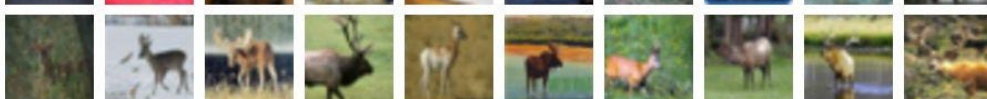
bird



cat



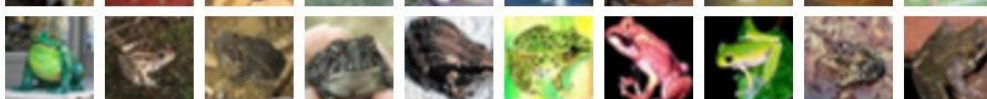
deer



dog



frog



horse



ship

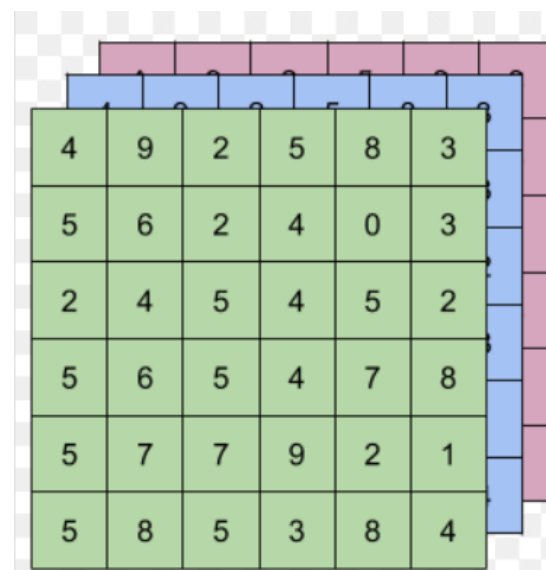


truck



50,000张训练图片， 每张
图片的大小为**32x32x3**。

10,000张测试图片。

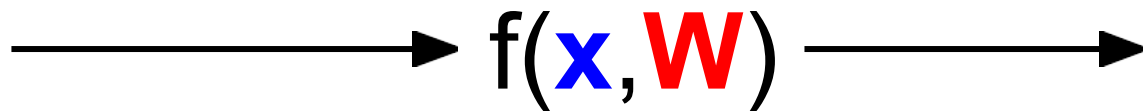


参数化方法

图片



包含**32x32x3**数字的数组
(总共3072个数字)



↑
W

参数或权重

10个数字表示类别得分。

参数化方法

$$f(x, W) = Wx$$

图片



包含**32x32x3**数字的数组
(总共3072个数字)



10个数字表示类别得分。

W

参数或权重

参数化方法：线性分类

图片



包含 $32 \times 32 \times 3$ 数字的数组
(总共3072个数字)

$$\boxed{f(x, W)} = Wx$$

10×1 10×3072 3072×1

$$\longrightarrow f(x, W) \longrightarrow$$

10个数字表示类别得分。

W

参数或权重

参数化方法：线性分类

图片



包含 $32 \times 32 \times 3$ 数字的数组
(总共3072个数字)

$$\boxed{f(x, W)} = Wx + \boxed{b}$$

10×1 10×3072 3072×1 10×1

→ $f(x, W)$ →

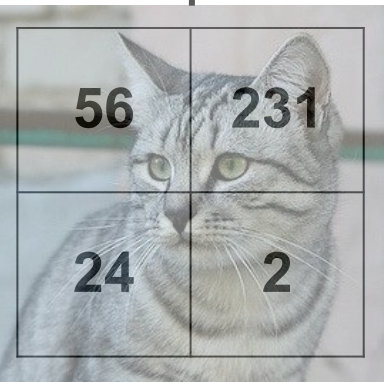
10个数字表示类别得分。

↑
 W

参数或权重

一个有4个像素的图片示例， 有3个类别 (cat/dog/ship)

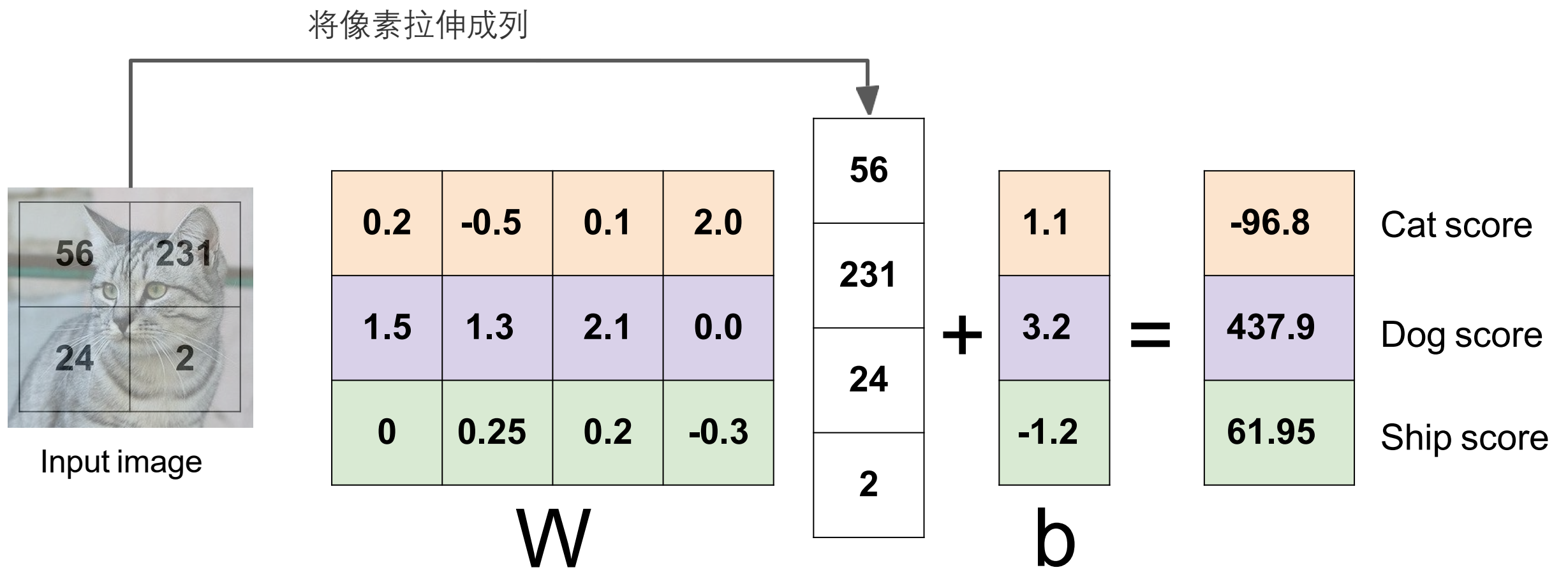
将像素拉伸成列



Input image



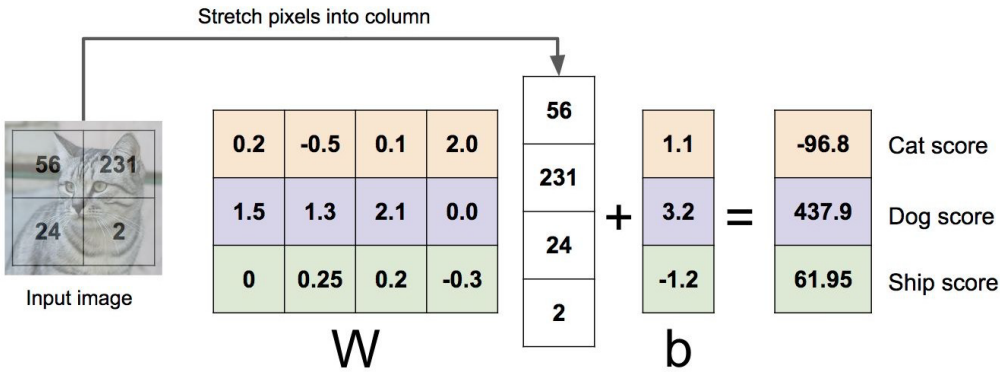
一个有4个像素的图片示例， 有3个类别 （猫/狗/船） (cat/dog/ship)



一个有4个像素的图片示例， 有3个类别（猫/狗/船） (cat/dog/ship)

代数视角

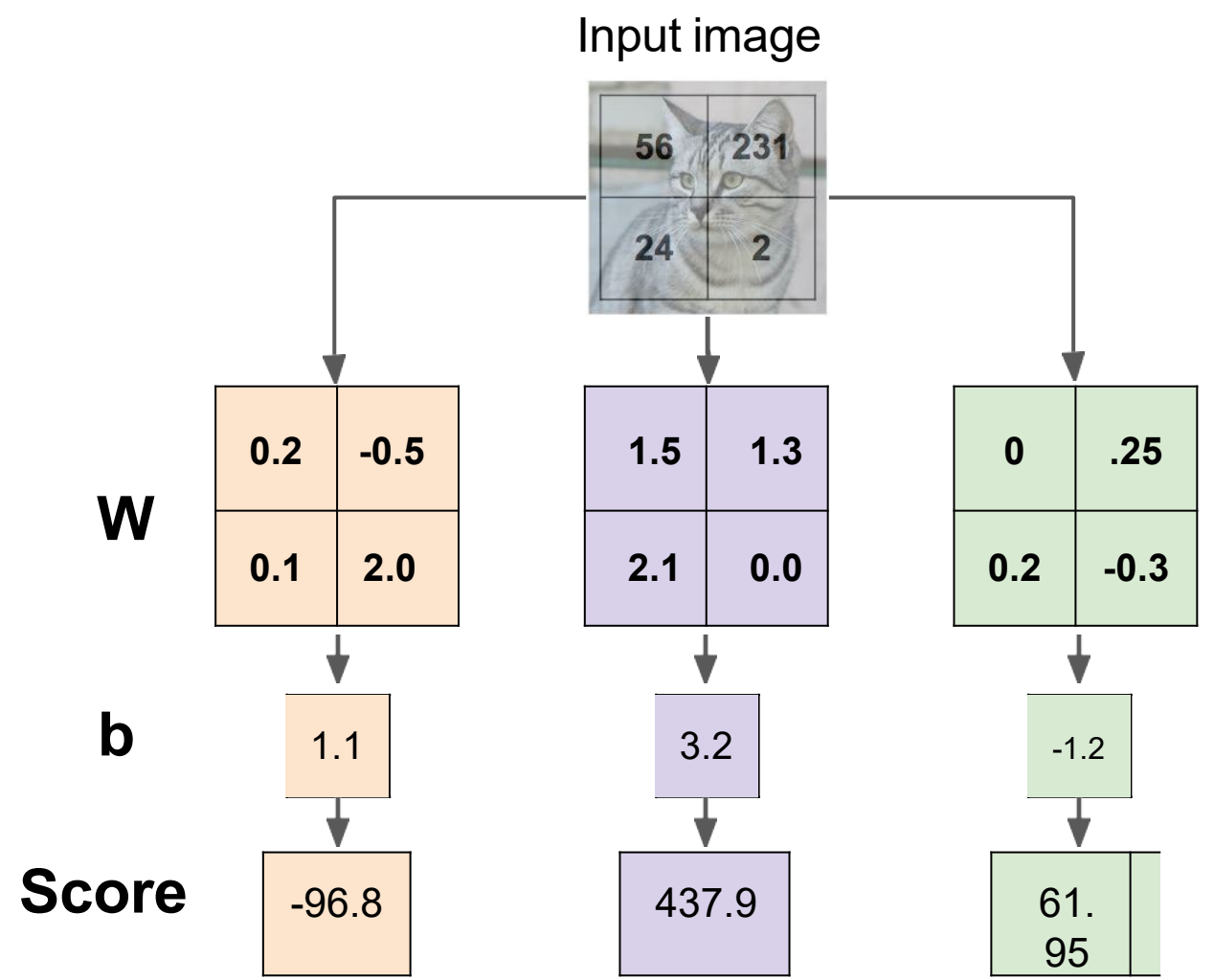
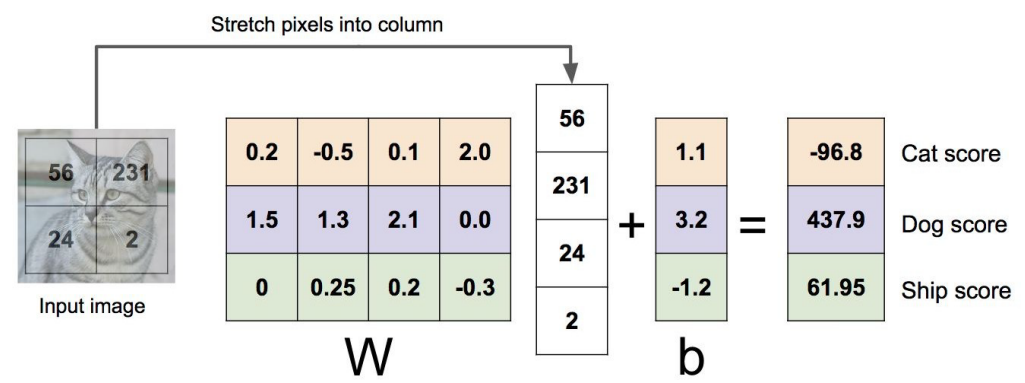
$$f(x,W) = Wx$$



一个有4个像素的图片示例， 有3个类别（猫/狗/船） (cat/dog/ship)

代数视角

$$f(x,W) = Wx$$



解释线性分类器

airplane



automobile



bird



cat



deer



dog



frog



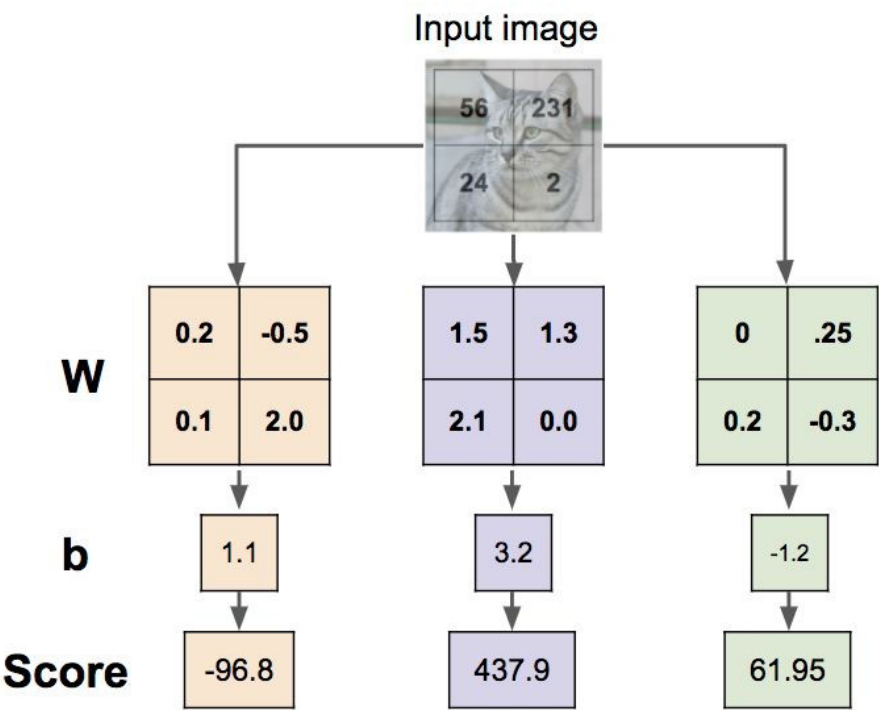
horse



ship



truck



解读线性分类器:视觉观点

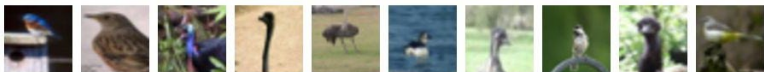
airplane



automobile



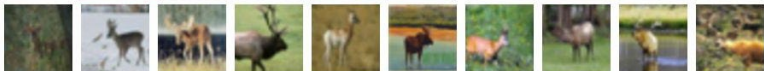
bird



cat



deer



dog



frog



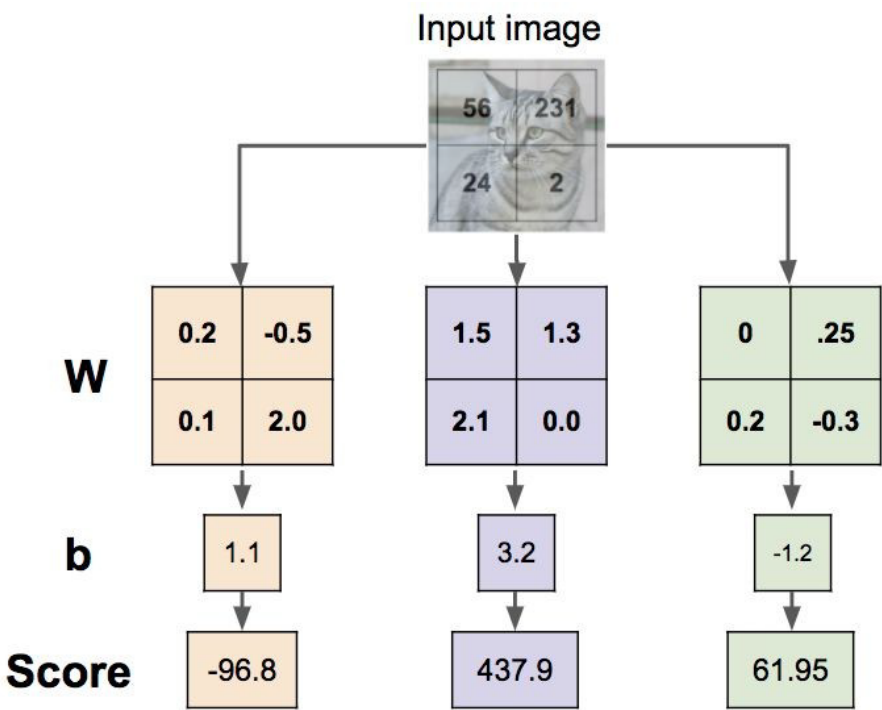
horse



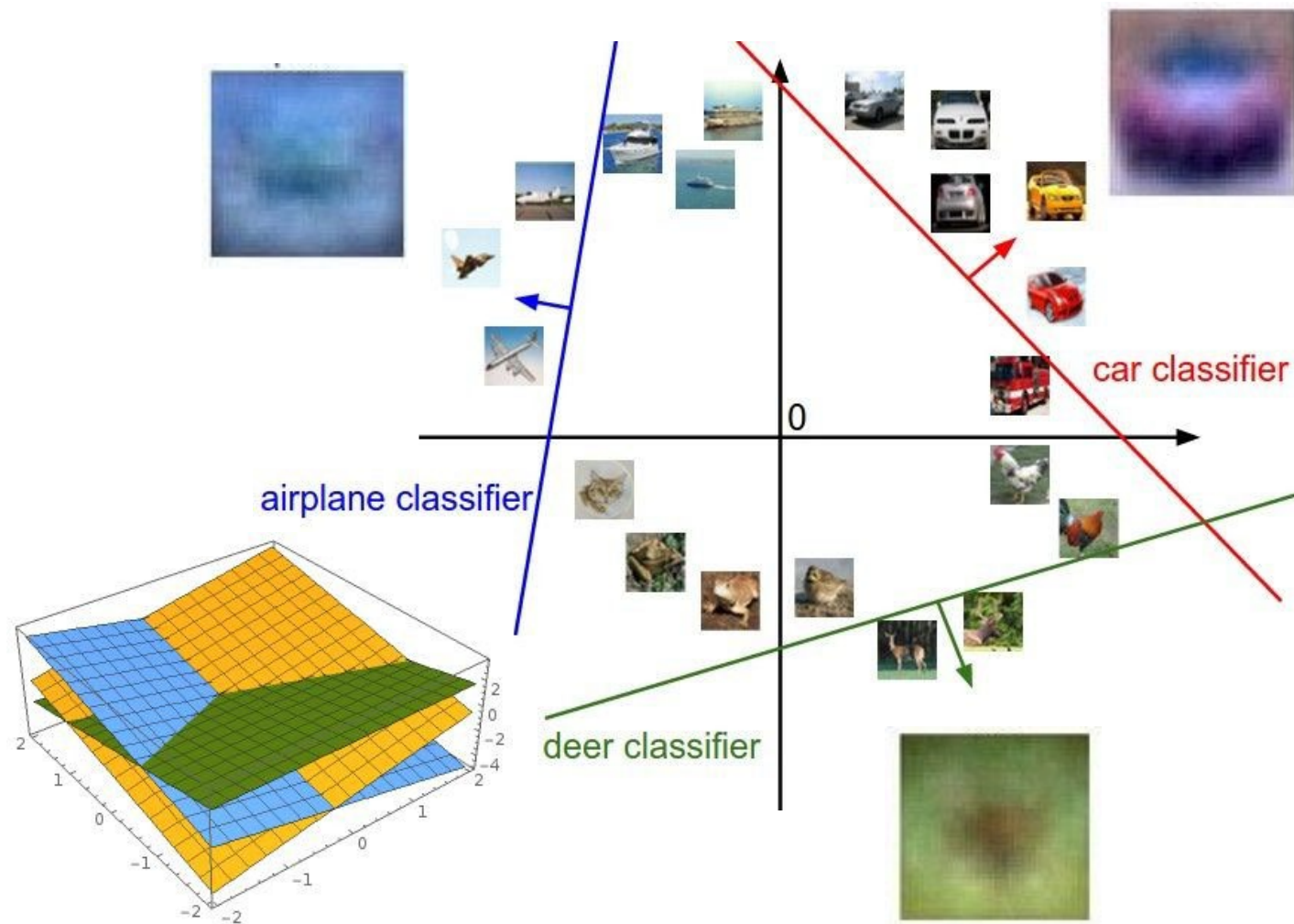
ship



truck



解释线性分类器:几何观点



$$f(x, W) = Wx + b$$



32x32x3数字的数组
(总计3072个数字)

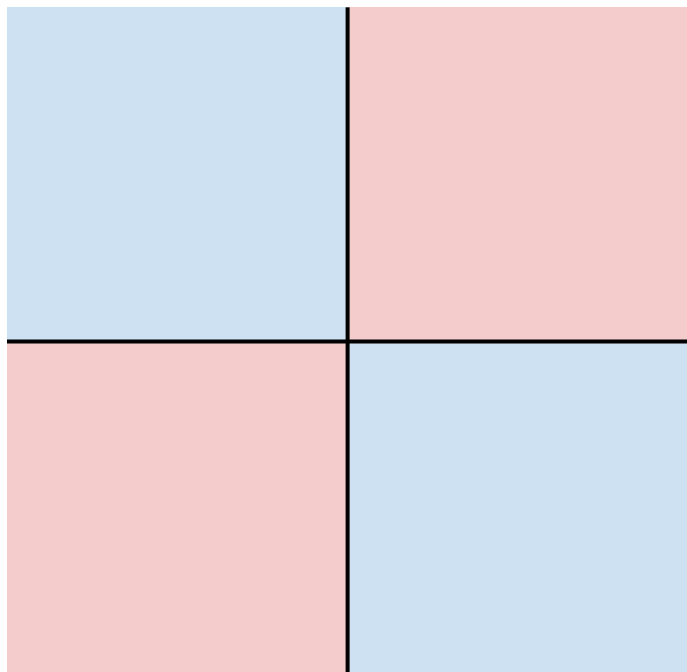
线性分类器的难题

Class 1:

第一和第三象限

Class 2:

第二和第四象限

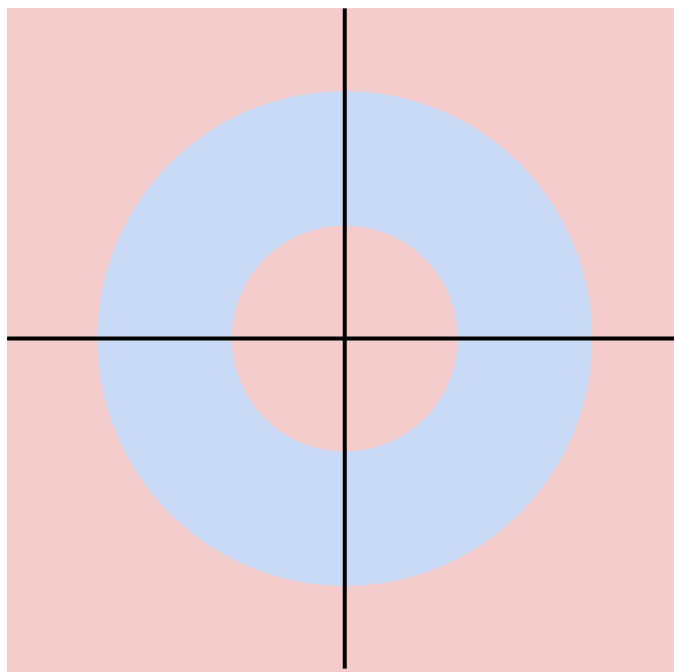


Class 1:

$1 \leq \text{L2 norm} \leq 2$

Class 2:

其他

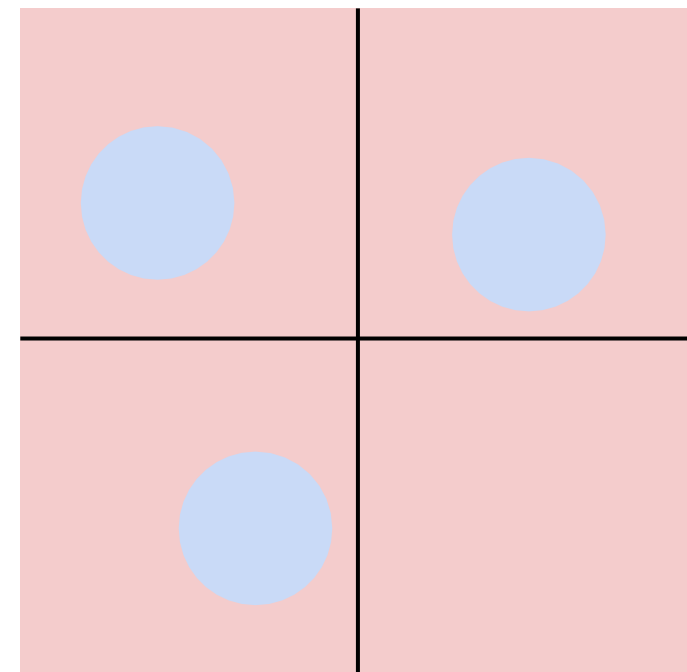


Class 1:

三个圆

Class 2:

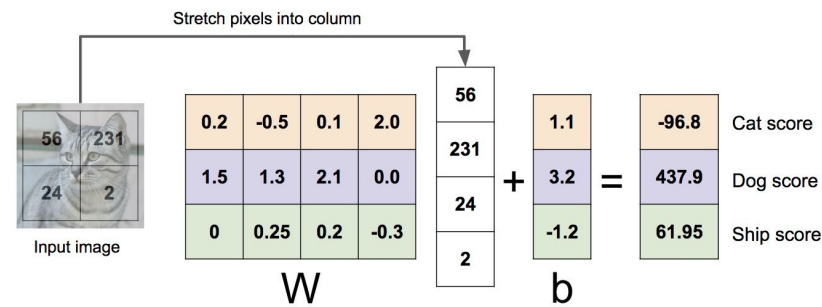
其他



Linear Classifier: Three Viewpoints

代数视角

$$f(x,W) = Wx$$



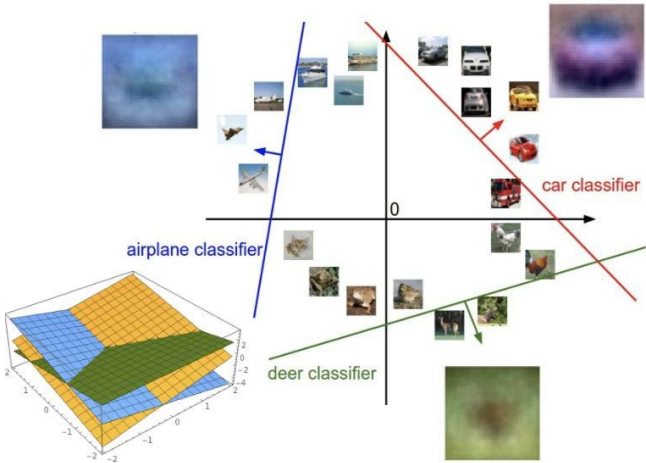
可视化视角

One template
per class



几何视角

Hyperplanes
cutting up space



至此:定义了一个(线性)分数函数 $f(x,W) = Wx + b$

某些W的3张图
像的示例分类
得分:



我们怎么知道这
个W是好是坏?

airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.31	2.65
cat	2.9	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

[Cat image](#) by [Nikita](#) is licensed under [CC-BY 2.0](#)
[Car image](#) is [CC0 1.0](#) public domain
[Frog image](#) is in the public domain